

VERIQO Customer Integration Guide

How to call the VERIQO Commercial API v1 from your own systems. The authoritative interface contract is [docs/customer/VERIQO_COMMERCIAL_API_V1_OPENAPI.yaml](#); this guide explains the *model* behind it, so your integration is shaped correctly rather than merely compiling.

1. The one thing to understand first

VERIQO does not store your evidence bytes. It stores each item's hash, provenance, chain of custody, and the decisions grounded on it. The bytes stay in your custody, at a URI you control.

This is deliberate. It means:

- Your document store, retention obligations, and access controls stay yours; VERIQO does not become a second copy of your sensitive data.
 - What VERIQO can prove is *integrity and lineage* -- that the artifact you present today is byte-identical to the one a decision was grounded on, and that its custody chain is unbroken.
 - **You must compute the SHA-256 yourself before submitting**, and you must keep the bytes. If you lose the bytes, VERIQO can still prove what the hash was and what was decided on it, but nobody can re-derive the document from the hash.
-

2. The integration shape

your system		VERIQO
-----		-----
compute sha256(bytes)	----->	POST /v1/evidence (metadata + hash + your URI) -> evidence finalized, under retention governance, custody chain started
open a matter	----->	POST /v1/cases
reach a conclusion	----->	POST /v1/cases/{id}/decide (cites evidence IDs; refused if any is not finalized)
act on it	----->	POST /v1/cases/{id}/actions (refused unless already decided)
export for a counterparty	----->	GET /v1/cases/{id}/dossier ?format=package -> a .zip your counterparty verifies WITHOUT trusting you or us

3. Logical time, not wall-clock time

Every `tick` / `*_at` field is a **caller-supplied logical clock**, not a timestamp VERIQO reads from the system. This is what makes replay deterministic: the same inputs always produce the same hashes, on any machine, at any

later date.

Practical guidance:

- Use a monotonically non-decreasing integer per case.
- A simple, working convention is a per-case counter (0, 10, 20, ...), leaving gaps so you can insert events later.
- If you need real-world time, record it in your own system and keep the mapping. Do not push epoch-milliseconds into `tick` unless you are certain it will remain non-decreasing across every actor writing to that case.

4. Tenancy and identity

`tenant_id` scopes everything. Under JWT authentication, the tenant you name **must** be one your authenticated subject is granted -- otherwise the request is refused with `403`, even with a valid token. You cannot act as an arbitrary tenant by typing its ID.

Two consequences worth designing around:

1. A resource belonging to another tenant returns **404, not 403**. Tenant scoping is structural (records are keyed by tenant), so a cross-tenant lookup never reaches an ownership check. Do not treat 404 as proof that an ID is unused globally.
2. If your deployment runs without JWT configured, `tenant_id` is taken as supplied. That mode is appropriate for a single-tenant or trusted-network deployment, not for multi-tenant production.

5. Errors you should handle deliberately

Status	Meaning	What to do
400	Malformed body, missing required field, or body over the size limit (1 MiB JSON / 64 MiB package).	Fix the request. Not retryable as-is.
403	Your subject may not act as this tenant, or the resource is another tenant's.	Do not retry. This is an authorization fact, not a transient failure.
404	No such case/evidence <i>for this tenant</i> .	Check the tenant as well as the ID.
409	Already exists (case), or already decided.	Decisions are once-only by design. If you need a different outcome, that is a new case, not a re-decision.
422	Well-formed but refused on the merits -- e.g. citing evidence that is not finalized, acting before deciding, exporting an undecided case.	Read the <code>error</code> string; it names the specific gate that refused.

Note that `POST /v1/evidence/{id}/verify` returning `{"verified": false}` is a **200**, not an error. A failed integrity check is a finding you must surface to a human, not an API fault.

6. Evidence must be grounded before it can be cited

`POST /v1/cases/{id}/decide` refuses to produce a Decision that cites evidence this deployment has not received and finalized. There is no override, no "trust me" flag, and no admin path around it.

This is the core guarantee you are buying: **a VERIQO Decision cannot reference evidence that does not exist**. Design your workflow so evidence submission genuinely precedes decision-making, rather than treating submission as an afterthought audit log.

Record contradicting evidence too. `contradicting_evidence_ids` exists because real cases have contradictions, and a dossier that shows them is more credible than one that quietly omits them.

7. Exporting for a counterparty

`GET /v1/cases/{id}/dossier?format=package` returns a `.zip` Machine Package. Hand it to your counterparty, their insurer, their lawyer, or an arbitrator along with the standalone verifier binary.

They run:

```
veriqo-commercial-verify -package case.zip [-trusted-keys keys.json]
```

The verifier is a **separate process** that reads only the `.zip` and, optionally, a trusted-key registry they obtained through their own channel. It does not call your servers or ours. That is the point: your counterparty does not have to trust you, or us, to check the package.

Exit code `0` = all checks passed; `1` = at least one check failed; `2` = usage/input error. See `docs/customer/VERIQO_UNIFIED_VERIFIER_SPECIFICATION.md`.

Without `-trusted-keys`, signature and key-state checks report `SKIP`, never `PASS`. If cryptographic authenticity matters to your counterparty, you must establish a key-distribution channel with them. See the Verifier Specification's "Trust anchor" section.

8. Retention and legal hold

Every submitted item is placed under retention governance at finalization. Evidence under legal hold cannot be purged -- the state machine has no edge from `HELD` to `PURGE_ELIGIBLE`, and the purge call independently refuses while a hold is present.

Current limitation, stated plainly: placing and releasing holds, setting retention policy, evaluating retention, and purging are reachable today only through the Go API, **not** through an HTTP route. If your pilot needs to drive holds from your own systems, raise it during scoping -- the capability is built and tested, only the HTTP surface is missing.

9. A minimal working sequence

```
BASE=https://veriqo.example.com
AUTH="-H 'Authorization: Bearer $TOKEN' -H 'Content-Type: application/json'"

# 1. Create the case
curl -X POST $BASE/v1/cases -d '{
  "tenant_id":"acme", "case_id":"CASE-1", "tick":0}'

# 2. Submit evidence you already hashed
curl -X POST $BASE/v1/evidence -d '{
  "tenant_id":"acme", "case_id":"CASE-1", "evidence_id":"EV-1",
  "sha256":"<64 hex chars you computed>",
  "uri":"s3://your-bucket/survey.pdf", "filename":"survey.pdf",
  "media_type":"application/pdf", "byte_size":48213,
  "collector":"independent-surveyor", "source":"surveyor-report",
  "domain":{"insurance":{"claim_id":"CLM-1", "evidence_kind":"SURVEY"}},
  "tick":10}'
```

```
# 3. Decide, citing that evidence
curl -X POST $BASE/v1/cases/CASE-1/decide -d '{
  "tenant_id":"acme",
  "hypothesis":{"id":"H1","description":"water ingress in transit"},
  "supporting_evidence_ids":["EV-1"],
  "finding_id":"F-1",
  "finding":{"contract_basis":"clause-4.2","obligation_ref":"OBL-1",
    "event_ref":"EVT-1","quantum_ref":"CALC-1",
    "human_review_required":true},
  "outcome":"APPROVED","rationale":"substantiated by finalized evidence",
  "ledger_actor":"adjuster-1","tick":20}'

# 4. Export a verifiable package
curl -o case.zip \
  "$BASE/v1/cases/CASE-1/dossier?tenant_id=acme&format=package"

# 5. Your counterparty verifies it, without trusting you
veriqo-commercial-verify -package case.zip
```

10. What to build on your side

- **Idempotency.** VERIQO refuses duplicate case creation (409) and re-decision (409). Treat both as success-if-already-done in your retry logic rather than as errors.
- **Hash discipline.** Compute the SHA-256 once, at acquisition, and store it alongside your bytes. Recomputing later from a file that has been through a converter or an email gateway is how integrity checks fail for uninteresting reasons.
- **Keep the dossier packages.** They are self-contained. A package exported today remains verifiable later without any VERIQO service being reachable.